

DFX Lab Food/Drink Monitor

User Manual and Technical Reference

DFX Capstone Team | University of South Florida | CIS 4910
Augusto Trufelman | Claudio Fischer | Damien Naha | Felipe Vignatti
Version 2.0 — April 2026

Table of Contents

Part 1: Technical Reference

1. Architecture Overview
2. Module Reference
3. Detection and Alert Logic
4. Advanced Detection Heuristics
5. HTTP API Reference
6. Settings and Validation
7. Frontend
8. Training Workflows
9. Constants Reference

1. Architecture Overview

The system is a multithreaded producer/consumer. A background camera worker thread captures frames from the primary camera, resizes them, runs YOLO inference, scores motion heuristics, applies the alert state machine, and publishes results to alerts.json plus cropped snippets to snippets/. A separate relay worker moves JPEG-encoded frames from the producer queue into a shared snapshot that the HTTP server reads for the MJPEG streams.

A ThreadingHTTPServer serves the dashboard HTML, MJPEG streams, alert JSON, settings, camera topology, and training endpoints. Because each HTTP connection is handled by its own thread, a slow browser client cannot block the inference loop and a slow inference pass cannot stall the MJPEG stream.

Shared state lives in a DashboardConfig instance (dfx/settings.py). HTTP handlers read and write DashboardConfig fields under a settings_lock so that browser settings changes take effect within one inference frame. Alert writes use a separate alert_lock.

A CameraManager (dfx/multicam.py) tracks the secondary preview camera in a CameraPreview worker thread. The secondary camera runs optional lightweight inference (for bounding-box overlays on its MJPEG stream) independently of the primary camera's full alert state machine.

2. Module Reference

2.1 dashboard.py

Entry point. Parses command-line arguments, calls configure_jetson_gpu_env(), loads the YOLO model via dfx.model_loader, builds the DashboardConfig, starts the frame relay and alert-persist background workers, starts the CameraManager for the secondary camera, starts the camera_worker thread, and launches the ThreadingHTTPServer. A StandaloneDashboardHandler subclass serves the inline HTML_PAGE on GET /.

2.2 main.py

Command-line-only webcam demo. Loads a YOLO model, opens the camera with cv2.VideoCapture, runs inference on every frame, filters results to FOOD_CLASS_NAMES, draws bounding boxes in an OpenCV window, and applies a simple persistence/cooldown state machine to write alerts to the JSON log. Primarily used for quick local testing without launching the full dashboard.

2.3 dfx/camera.py

The camera_worker function runs in a dedicated thread. It opens the camera with a fallback open sequence (trying multiple backends), reads frames in a tight loop, flips them 180° (cameras are mounted upside-down), resizes to the configured dimensions, runs YOLO inference via predict_with_fallback, separates food detections from person detections, calls detect_consumption_motion and detect_person_hand_to_mouth_proxy from dfx/motion.py, feeds results into the alert state machine, writes approved alerts via dfx/alerts.py, and enqueues JPEG-encoded frames for the relay worker.

2.4 dfx/alerts.py

Handles all alert I/O. Provides read_alerts and write_alerts for the JSON log, append_alert for atomic alert creation, save_alert_snippets for cropping and saving bounding-box images, append_detection_summary_csv for per-detection CSV rows, and the suppression helpers

is_same_person_alert and is_new_object_alert. Also provides export_accepted_alert_samples which exports accepted-alert snippets into the training dataset.

2.5 dfx/motion.py

Heuristic motion scoring for eating/drinking detection. detect_consumption_motion maintains a per-object deque of (x, y, width, height, timestamp) samples, smooths the trajectory, computes normalized displacement and upward component, and produces a scalar motion score.

detect_person_hand_to_mouth_proxy uses MediaPipe Holistic to detect wrist-to-mouth 2D proximity: it crops the best person bounding box, pads it to a square, runs pose landmark inference, computes the normalized wrist-to-mouth distance and dwell time, and returns a boolean + score when the landmark geometry matches a hand-to-mouth gesture.

2.6 dfx/multicam.py

CameraPreview is a worker thread for one secondary camera. It opens the camera, reads frames in a loop, flips them, optionally runs lightweight inference for bounding-box overlays, JPEG-encodes the result, and exposes get_frame() / get_jpeg() for the HTTP server to poll. CameraManager tracks a dict of active CameraPreview workers, propagates settings updates to all of them, and provides add_camera / remove_camera for runtime topology changes.

2.7 dfx/server.py

DashboardHandler subclasses BaseHTTPRequestHandler. It implements do_GET and do_POST routing for all API endpoints, the MJPEG streaming loop, snapshot serving, camera topology responses, and the two-camera fixed layout (_FIXED_PRIMARY_CAMERA_INDEX = 0, Zone A; _FIXED_SECONDARY_CAMERA_INDEX = 2, Zone F). It maintains and persists dashboard_state.json (zone counts and processed alert IDs) and applies camera-zone tag mapping to incoming alerts.

2.8 dfx/settings.py

DashboardConfig is a plain Python dataclass that holds every runtime-tunable parameter plus shared mutable state (latest frames, alert lock, training lock, motion tracks, etc.). Helpers parse_bool, normalize_camera_zone, clamp_float, and clamp_int validate and bound settings values received from the browser. settings_snapshot returns the current config as a JSON-serializable dict for GET /settings.

2.9 dfx/gpu.py

Detects whether the host is a Jetson device (is_jetson_linux), sets up the CUDA library path (configure_jetson_gpu_env), selects the best available inference device (get_best_device: CUDA > MPS > CPU), wraps YOLO inference with graceful fallback on OOM errors (predict_with_fallback), and optionally exports a TensorRT engine for faster Jetson inference.

2.10 dfx/model_loader.py

load_inference_model loads a YOLO model from disk, verifies the weights file exists, and moves the model to the selected device. Supports hot-swapping: the running dashboard can reload a newly trained best.pt without restarting by calling this function and replacing the model reference in DashboardConfig under model_lock.

2.11 dfx/detection.py

`detections_from_result` converts a raw Ultralytics Results object into a list of plain dicts with `class_id`, `class_name`, `confidence`, `bbox_xyxy`, and `center_xy` fields. `draw_detections` renders orange bounding boxes and class/confidence labels onto a frame copy using `cv2.rectangle` and `cv2.putText`.

2.12 dfx/constants.py

Single source of truth for all tunable constants, class-name sets, default paths, and fixed camera topology labels. Importing from constants guarantees consistency across all modules without circular imports.

2.13 dfx/training.py

`_train_on_accepted_samples` runs as a background thread: it suspends detection (`config.detection_enabled = False`), moves the inference model to CPU, exports accepted snippets, builds a YOLO dataset YAML, calls `model.train()` with Jetson-safe overrides when needed, and hot-swaps `best.pt` back into the running config. `refresh_runtime_class_names` reads the new `class_map.json` and updates the config so inference uses the expanded vocabulary immediately.

2.14 train_local_kaggle.py

Standalone offline training script, independent of the dashboard flow. Accepts either a local dataset directory or a Kaggle dataset URL/slug. Downloads the Kaggle dataset via the Kaggle CLI if needed, builds a YOLO-format dataset with a configurable val split, optionally collapses all labels to a single class or preserves existing model classes, writes `data.yaml`, and calls `model.train()`. Output is written to `training_data/runs/accepted/weights/best.pt` by default, which is the same path the dashboard startup prefers.

3. Detection and Alert Logic

Both `main.py` and the `camera_worker` in `dashboard.py` use a three-stage state machine to decide when to fire an alert.

3.1 Class Filtering

YOLO is called with the union of `FOOD_CLASS_NAMES` and `'person'` (`INFERENCE_CLASS_NAMES`). The results are split: food detections go to the alert state machine, person detections go to the hand-to-mouth proxy pipeline. Detections below `ALERT_DETECTION_CONFIDENCE_FLOOR` (0.30) are dropped before alert evaluation; detections below `ALERT_SNIPPET_CONFIDENCE_FLOOR` (0.25) are dropped before snippet saving.

3.2 Persistence Window

The loop maintains a consecutive-frame counter. Every frame with at least one surviving food detection increments it; every empty frame resets it. An alert is only eligible to fire once the counter reaches `--persist-frames` (default 5). This eliminates single-frame false positives from motion blur or compression noise.

3.3 Cooldown and Re-Arm

When an alert fires, `last_alert_ts` is recorded and the state machine disarms. It re-arms once two conditions are simultaneously true: `--cooldown` seconds have elapsed since the last alert, and at least `--clear-frames` consecutive empty frames have been seen. This prevents a stationary bottle from generating thousands of alerts while still allowing a fresh detection after the scene has cleared to fire immediately.

3.4 Stationary Follow-up

A stationary food item (one that was detected once but is no longer generating active motion scores) generates one follow-up alert after `STATIONARY_FOLLOWUP_SECONDS` (30 minutes). This satisfies the requirement that a soda can left on a desk receives a reminder alert after 30 minutes.

3.5 New-Object vs. Same-Person Distinction

`is_new_object_alert` checks whether the current alert's bounding box overlaps a recent alert within `NEW_OBJECT_LOOKBACK_SECONDS` (30 s). If not, it is treated as a new object and fires unconditionally. `is_same_person_alert` checks whether a recent alert at nearly the same location (`SAME_PERSON_SUPPRESSION_DISTANCE_RATIO` = 0.18 of bounding-box diagonal) was issued within `SAME_PERSON_ALERT_WINDOW_SECONDS` (120 s) and fewer than `SAME_PERSON_MAX_ALERTS_IN_WINDOW` (3) such alerts have been issued. If yes, the new alert is suppressed to prevent alert spam from a single person eating continuously.

4. Advanced Detection Heuristics

`dashboard.py` layers a set of heuristics on top of the basic state machine to reduce false positives and to distinguish 'someone is carrying food' from 'someone is actively eating'. All heuristics are toggled by `--motion-enabled` / `--no-motion-enabled` and the Settings panel. Constants live in `dfx/constants.py`.

4.1 Consumption Motion Analysis

For each tracked food object, the system keeps a sliding deque of (x, y, diagonal, timestamp) samples of length `--motion-window` (default 4). On each frame it computes an exponentially smoothed trajectory ($\alpha = 0.45$), the normalized displacement, and the normalized upward component. Displacements above `--motion-displacement-threshold` combined with an upward component above `--motion-upward-threshold` are scored as consumption motion. An object does not count as 'being consumed' until it accumulates `FOOD_MOTION_MIN_SCORE` (1.0) and remains above `MOTION_TRIGGER_SCORE` (0.85) for `FOOD_MOTION_CONFIRM_FRAMES` (3) frames. Positive signals are held active for `--motion-hold-seconds` (0.1 s) to bridge brief occlusions.

4.2 Hand-to-Mouth Events (legacy heuristic)

A hand-to-mouth event is a local peak in the upward-biased motion score that looks like a bite or sip. The system looks for `HAND_TO_MOUTH_REQUIRED_EVENTS` (3) such events within a rolling `HAND_TO_MOUTH_WINDOW_SECONDS` (30 s) window before upgrading a detection from 'present' to 'actively being consumed'. Individual event thresholds are `FOOD_HAND_TO_MOUTH_EVENT_MIN_SCORE` (1.4) for direct food detections and `PROXY_HAND_TO_MOUTH_EVENT_MIN_SCORE` (1.2) for person-proxy events.

4.3 MediaPipe Pose Landmark Detection (strict hand-to-mouth)

When MediaPipe is installed, `detect_person_hand_to_mouth_proxy` uses the Holistic model to run 2D pose landmark inference on the best person crop each frame. The crop is padded to a square before inference so MediaPipe's internal ROI calculations remain stable. The wrist landmarks (indices 15 and 16) are projected into full-frame coordinates and their distance from the mouth center landmarks (indices 9 and 10) is normalized by the person bounding-box scale. A proximity event fires when:

- The normalized wrist-to-mouth distance is below `HAND_MOUTH_MAX_DISTANCE_RATIO` (0.92).
- The wrist has been within range for at least `HAND_MOUTH_MIN_DWELL_SECONDS` (0.26 s).
- The wrist is approaching the mouth (delta ratio above `HAND_MOUTH_MIN_APPROACH_DELTA_RATIO`, direction cosine above `HAND_MOUTH_MIN_DIRECTION_COSINE`).
- The wrist has moved upward by at least `HAND_MOUTH_MIN_WRIST_UPWARD_RATIO` (0.035) over `HAND_MOUTH_MIN_WRIST_UPWARD_STEPS` (2) recent steps within `HAND_MOUTH_WRIST_UPWARD_HISTORY_SECONDS` (0.80 s).

A confirmed event sets `active_until` = now + `HAND_MOUTH_HOLD_SECONDS` (0.60) and returns score `HAND_MOUTH_SCORE_FLOOR` (1.24). EMA smoothing ($\alpha = 0.33$) is applied to landmark positions to reduce jitter.

4.4 Person-Proxy Fallback (food occlusion)

When a food item is temporarily hidden behind a hand or phone, the system falls back to tracking the person detection near where the food was last seen. A person detection within `FOOD_OCCLUSION_LOOKBACK_SECONDS` (2.0 s) of the last food sighting becomes a proxy. Within `OCCLUDED_MOTION_HOLD_SECONDS` (1.2 s), an upward movement of the proxy above `OCCLUDED_MOTION_PROXY_SCORE` (0.86) is counted as a consumption event attributed to the hidden food. The proxy must cover at least `PERSON_PROXY_MIN_AREA_RATIO` (0.02) of the frame and pass mouth-region motion and approach thresholds before it can contribute to an alert.

4.5 Same-Person Alert Suppression

Described in section 11.5. Constants: `SAME_PERSON_ALERT_WINDOW_SECONDS` = 120, `SAME_PERSON_MAX_ALERTS_IN_WINDOW` = 3, `SAME_PERSON_SUPPRESSION_DISTANCE_RATIO` = 0.18.

5. HTTP API Reference

All endpoints are served by the `ThreadingHTTPServer` bound to `--host:--port` (default 0.0.0.0:8000).

5.1 GET Endpoints

Endpoint	Description
GET /	Returns the embedded HTML dashboard page.
GET /stream	MJPEG multipart stream of annotated frames from the primary camera.

Endpoint	Description
GET /stream?camera=2	MJPEG multipart stream from the secondary preview camera.
GET /snapshot	Single JPEG frame from the primary camera.
GET /snapshot?camera=2	Single JPEG frame from the secondary camera.
GET /alerts?limit=N	Last N alerts from alerts.json as JSON. Limit is clamped 0–500.
GET /settings	Current DashboardConfig as JSON.
GET /cameras	Fixed two-camera topology with zone assignments.
GET /classes	Merged list of known food/drink class names.
GET /train/status	Training state snapshot (running, last run, accepted count).
GET /stats/consumption	Alert counts grouped by zone and item category.
GET /map	Returns the lab layout image if the file exists.
GET /snippets/<filename>	Returns a saved cropped-bounding-box image.

5.2 POST Endpoints

Endpoint	Body	Description
POST /alerts/manage	JSON: {id, status}	Update alert status (reviewed/dismissed/accepted).
POST /settings	JSON: settings fields	Apply runtime settings. Fields are validated and clamped.
POST /train/start	Empty or {}	Start a background fine-tuning job on accepted samples.
POST /train/accept	JSON: {alert_id}	Mark one alert as accepted (alias for manage).

6. Settings and Validation

Every numeric field received via POST /settings is validated and clamped by helpers in dfx/settings.py before it is written into DashboardConfig:

- `clamp_float(value, field_name, min, max)` — parses the value as a float and clamps it to `[min, max]`.
- `clamp_int(value, field_name, min, max)` — parses the value as an int and clamps it to `[min, max]`.
- `parse_bool(value, field_name)` — accepts true/false, 1/0, yes/no, on/off.
- `normalize_camera_zone(value)` — accepts 'Zone A' through 'Zone I' in any case variation.

Values outside the allowed range are rejected with a 400 response so a browser typo cannot put the system into a bad state. Settings take effect within the next inference frame because the `camera_worker` reads `DashboardConfig` fields under `settings_lock` on every iteration.

7. Frontend

The dashboard frontend is a single inline HTML page stored in `dfx/server.py` as the constant `HTML_PAGE` and served on `GET /`. It is intentionally dependency-free: no build step, no npm, no frontend framework. The page uses vanilla JavaScript to:

- Poll `GET /alerts` every few seconds and render alert cards.
- POST state changes to `/alerts/manage`.
- Fetch and submit runtime settings against `/settings`.
- Render two live MJPEG feeds inside `<img src='/stream'>` and `<img src='/stream?camera=2'>` tags.
- Display training progress from `GET /train/status`.

The CSS, alert card renderer, zone grouping code, settings form logic, map view, and training progress view are all embedded in the same string. This keeps deployment to a single copy of `server.py` plus model weights.

8. Training Workflows

8.1 Dashboard Accept/Fine-tune Flow

1. Mark alerts as Accepted in the dashboard.
2. Press Train. The training worker exports cropped snippets from accepted alerts into `training_data/dataset/`, writes `data.yaml`, fine-tunes the loaded model for `--train-epochs epochs`, and saves `best.pt` to `training_data/runs/accepted/weights/`.
3. The new model is hot-swapped into the running dashboard. `refresh_runtime_class_names` reads `class_map.json` and expands the detection vocabulary if new classes were added.

8.2 Offline Kaggle / Local Training

Use `train_local_kaggle.py` for bulk training from external datasets without the dashboard UI:

```
python train_local_kaggle.py --dataset-dir /path/to/dataset --class-names  
canned_drink
```

To collapse all source labels to a single class:

```
python train_local_kaggle.py --dataset-dir /path/to/dataset \  
--collapse-to-single-class --single-class-name soda_can
```

To add a new class while preserving the existing model vocabulary:

```
python train_local_kaggle.py --dataset-dir /path/to/dataset \  
--model training_data/runs/accepted/weights/best.pt \  
--collapse-to-single-class --single-class-name soda_can \  
--preserve-existing-classes
```

To train directly from a Kaggle dataset URL:

```
python train_local_kaggle.py \  
--kaggle-dataset https://www.kaggle.com/datasets/<owner>/<dataset> \  
--class-names canned_drink
```

Kaggle prerequisites: install the Kaggle CLI (`pip install kaggle`) and place your API token at `~/.kaggle/kaggle.json`.

8.3 Jetson VRAM Management

Training on the Jetson Orin Nano's shared GPU is constrained by VRAM. The dashboard training worker automatically moves the live inference model to CPU before training begins and restores it to GPU afterward. `train_local_kaggle.py` sets the PyTorch CUDA async allocator environment variable before any torch import and applies Jetson-safe training overrides (SGD optimizer, disabled mosaic/mixup augmentation, frozen backbone) when `--no-cudnn` is passed. If a CUDA allocator error is still detected, training is automatically retried on CPU.

9. Key Constants Reference

All constants in `dfx/constants.py` act as the single source of truth for tuning the heuristics. They can be changed without touching the inference loop.

Class-Name Sets

Constant	Description
FOOD_CLASS_NAMES	COCO food/drink classes plus SNACK_CLASS_NAMES (chip bags, candy, vending-machine food, etc.).
INFERENCE_CLASS_NAMES	FOOD_CLASS_NAMES U {'person'}. Passed to YOLO so persons participate in proximity heuristics.
CONSUMPTION_CLASS_NAMES	Subset of food classes that can trigger consumption alerts (excludes 'bowl').
HANDHELD_FOOD_CLASS_NAMES	Food classes typically held in hand. Used by motion grouping to avoid resetting the tracker when the class label flickers.
DRINK_CONTAINER_CLASS_NAMES	{'bottle', 'cup', 'drink', 'drinks'}. Grouped by <code>_consumption_track_key</code> to prevent tracker resets.

Motion Scoring

Constant	Value	Description
MOTION_TRIGGER_SCORE	0.85	Minimum score for a consumption motion signal to be considered active.
FOOD_MOTION_MIN_SCORE	1.0	Accumulated score required before consumption motion is confirmed.
FOOD_MOTION_CONFIRM_FRAMES	3	Consecutive frames above MOTION_TRIGGER_SCORE required to confirm motion.
FOOD_HAND_TO_MOUTH_EVENT_MIN_SCORE	1.12	Score threshold for a direct food hand-to-mouth event.
PROXY_HAND_TO_MOUTH_EVENT_MIN_SCORE	1.2	Score threshold for a person-proxy hand-to-mouth event.
HAND_TO_MOUTH_WINDOW_SECONDS	30.0	Rolling window for counting hand-to-mouth events.
HAND_TO_MOUTH_REQUIRED_EVENTS	3	Events required within the window to upgrade to 'actively consuming'.

Constant	Value	Description
STATIONARY_FOLLOWUP_SECONDS	1800	Seconds before a stationary item generates a follow-up alert.

MediaPipe Landmark Parameters

Constant	Value	Description
HAND_MOUTH_MAX_DISTANCE_RATIO	0.92	Max normalized wrist-to-mouth distance ratio.
HAND_MOUTH_MIN_DWELL_SECONDS	0.26	Minimum dwell time in proximity before event fires.
HAND_MOUTH_HOLD_SECONDS	0.60	How long an active landmark event stays asserted.
HAND_MOUTH_SCORE_FLOOR	1.24	Score returned when a landmark event is active.
HAND_MOUTH_LANDMARK_EMA_ALPHA	0.33	EMA smoothing factor for landmark positions.
HAND_MOUTH_MIN_WRIST_UPWARD_RATIO	0.035	Minimum upward wrist displacement ratio.
HAND_MOUTH_MIN_WRIST_UPWARD_STEPS	2	Minimum upward steps in recent history.

Alert Confidence Floors

Constant	Value	Description
ALERT_DETECTION_CONFIDENCE_FLOOR	0.30	Minimum detection confidence for alert evaluation.
ALERT_SNIPPET_CONFIDENCE_FLOOR	0.25	Minimum detection confidence for snippet saving.
HAND_TO_MOUTH_FOOD_VISIBILITY_FLOOR	0.45	Minimum confidence for a food item to count as 'visible' in hand-to-mouth analysis.

Suppression and New-Object

Constant	Value	Description
SAME_PERSON_ALERT_WINDOW_SECONDS	120.0	Window for same-person suppression.
SAME_PERSON_MAX_ALERTS_IN_WINDOW	3	Max alerts from one location before suppression.
SAME_PERSON_SUPPRESSION_DISTANCE_RATIO	0.18	Bounding-box distance ratio defining 'same location'.
NEW_OBJECT_LOOKBACK_SECONDS	30.0	How far back to look when deciding if an alert is a new object.

Constant	Value	Description
NEW_OBJECT_MATCH_DISTANCE_RATIO	0.25	Distance ratio for matching a detection to a past alert.
NEW_OBJECT_MIN_CONFIDENCE	0.30	Minimum confidence for a new-object alert.

Fixed Camera Topology

Constant	Value	Description
DASHBOARD_PRIMARY_CAMERA_INDEX	0	Device index for the primary inference camera.
DASHBOARD_PRIMARY_CAMERA_ZONE	Zone A	Zone label for the primary camera.
DASHBOARD_SECONDARY_CAMERA_INDEX	2	Device index for the secondary preview camera.
DASHBOARD_SECONDARY_CAMERA_ZONE	Zone F	Zone label for the secondary camera.